

This is the ultimate roadmap for your project. By implementing the upgraded features we discussed, you guarantee that each member's workload matches the rigorous complexity of [Assignment_3.py](#).

Here is the complete, detailed breakdown and the requested text files to set your team—and your AI assistants—up for absolute success.

Part 1: Finalized Feature List & Implementation Guide

To pass the complexity check, your "Obstacle Runner" needs these specific, upgraded features.

Member 1 Features: Environment & Camera

- **3D Scrolling Arcade Tunnel:** * *What it does:* Instead of a flat floor, the player runs through a neon tunnel or city street.
 - *How to build it:* Draw a floor grid using [GL_QUADS](#). On the edges (e.g., $x = -150$ and $x = 150$), render a series of pillars using stacked [glutSolidCube](#) and [gluCylinder](#). Translate all these objects along the Z-axis every frame. When a pillar passes $Z = 0$, snap its position back to the far end of the tunnel.
- **Day/Night Cycle (Dynamic Lighting):**
 - *What it does:* The environment's colors slowly shift as the player survives longer.
 - *How to build it:* Use a global [time_survived](#) variable to calculate an interpolation factor. Pass this factor into your [glColor3f\(R, G, B\)](#) calls to smoothly transition the grid from bright daytime colors to dark neon colors.
- **Warp Speed Camera (Novelty):**
 - *What it does:* Creates a visual distortion as the game speeds up.
 - *How to build it:* Inside [setupCamera\(\)](#), dynamically adjust the [fovY](#) parameter in [gluPerspective](#). As the global speed increases, increase [fovY](#) from 120 to 150.

Member 2 Features: Player Controller & Physics

- **Hierarchical Animated Avatar:**
 - *What it does:* A multi-part runner (like the robot in Assignment 3) rather than a single cube.
 - *How to build it:* Use nested [glPushMatrix](#) and [glPopMatrix](#) to attach limbs to a central body. Use a [sin\(\)](#) wave tied to the game time to rotate the arms and legs back and forth ([glRotatef](#)) when in the "running" state.

- **Smooth Lane-Switching & Kinematics:**

- *What it does:* The avatar smoothly glides between lanes and jumps using real gravity math.
- *How to build it:* For X-axis movement, interpolate the current X to the target lane X over a few frames. For jumping, use the kinematic equation $y = v_{0t} - \frac{1}{2}gt^2$ to update the Y-coordinate.

- **Gravity Inversion (Novelty):**

- *What it does:* Flips the player to run on the ceiling.
- *How to build it:* Toggle a `gravity_inverted` boolean. When true, instantly translate the player's base Y-coordinate to the ceiling height, invert the jump gravity constant g , and rotate the avatar 180 degrees around the Z-axis so it runs upside down.

Member 3 Features: Obstacle Engine & Collisions

- **Pattern-Based Spawning & Collectibles:**

- *What it does:* Spawns obstacles in deliberate, challenging formations alongside collectible power-ups (like coins or gravity-flip items).
- *How to build it:* Create lists of dictionaries for `obstacles` and `powerups`. Write a state machine that randomly selects a "formation" (e.g., three low blocks, followed by one high block) and populates the arrays at the far end of the Z-axis.

- **Morphing/Pulsing Logic (Novelty):**

- *What it does:* Obstacles animate or change shape as they approach.
- *How to build it:* In the update loop, if an obstacle's Z-distance is less than a specific threshold, alter its `glScalef` values to make it pulse, or swap its rendering function from `glutSolidCube` to `gluSphere`.

- **Dynamic 3D AABB Collision:**

- *What it does:* Checks if the player hits an obstacle or collects a power-up.
- *How to build it:* Implement Axis-Aligned Bounding Box math. Crucially, the player's bounding box height must change dynamically depending on whether Member 2's avatar is running, ducking, or inverted.

Github task sequence:

Here is the exact GitHub task sequence your team must follow. Since your professor requires the entire project to be in a single `.py` file, you are at a very high risk of Git merge conflicts.

To prevent this, you will use a "**Zoned Coding**" strategy combined with a strict sequential merging process for the core engine, followed by parallel merging for the novelty features.

The "Zoned Coding" GitHub Sequence

Phase 0: Repository Initialization & Zoning

This phase must be completed before anyone writes a single line of game logic.

1. **Member 1 (Repository Lead)** creates the GitHub repository and uploads the base `3D_OpenGL_Intro.py` template.
2. **Member 1** opens the file and creates "Zones" using massive comment blocks. These are strict boundaries. No one is allowed to write functions outside their assigned zone.
 - `# === ZONE 0: GLOBAL VARIABLES ===`
 - `# === ZONE 1: MEMBER 1 (WORLD & CAMERA) ===`
 - `# === ZONE 2: MEMBER 2 (PLAYER CONTROLLER) ===`
 - `# === ZONE 3: MEMBER 3 (OBSTACLES & COLLISIONS) ===`
 - `# === ZONE 4: MAIN GAME LOOP ===`
3. **Member 1** commits and pushes this zoned file to the `main` branch.
4. **All Members** clone the repository to their local machines.

Phase 1: The Environment (Member 1)

The game needs a world before the player has somewhere to run.

1. **Member 1** creates a new branch: `git checkout -b feature-world`.
2. **Member 1** codes the scrolling 3D arcade tunnel, day/night color interpolation, and the basic camera setup *strictly inside Zone 1*.
3. **Member 1** tests the code, commits, and pushes the branch.

4. **Member 1** creates a Pull Request (PR) and merges `feature-world` into `main`.

Phase 2: The Player (Member 2)

Now that the tunnel is moving, the player avatar can be added.

1. **Member 2** pulls the updated main branch: `git checkout main -> git pull`.
2. **Member 2** creates a new branch: `git checkout -b feature-player`.
3. **Member 2** codes the hierarchical animated avatar, lane switching, and jumping kinematics *strictly inside Zone 2*.
4. **Member 2** tests their player inside Member 1's moving tunnel, commits, and pushes.
5. **Member 2** creates a PR and merges `feature-player` into `main`.

Phase 3: The Threat (Member 3)

With the player running in the tunnel, obstacles can now be spawned and collided with.

1. **Member 3** pulls the updated main branch: `git checkout main -> git pull`.
2. **Member 3** creates a new branch: `git checkout -b feature-obstacles`.
3. **Member 3** codes the pattern-based obstacle spawning, collectibles, and dynamic 3D AABB collision detection *strictly inside Zone 3*.
4. **Member 3** tests that the obstacles successfully hit Member 2's avatar, commits, and pushes.
5. **Member 3** creates a PR and merges `feature-obstacles` into `main`.

At this point, the core game is 100% playable.

Phase 4: Novelty Integration (Parallel Workflow)

Now that the core engine is stable and everyone respects their Zones, the team can work simultaneously on their bonus features.

1. **All Members** pull the latest `main` branch.
2. **All Members** branch off at the same time:
 - Member 1: `git checkout -b feature-warp-speed`
 - Member 2: `git checkout -b feature-gravity-inversion`
 - Member 3: `git checkout -b feature-morphing-obstacles`
3. Because the core functions are already built and everyone is working strictly inside their own comment zones, you will not overwrite each other's code.

4. As each member finishes their novelty feature, they can independently open a PR and merge it back into `main`. Resolve any minor global variable conflicts together as a team.

AI CONTEXT FILES I HAVE PROVIDED IN THE GROUP SO IGNORE PLEASE AND USE THESE TO GIVE CONTEXT TO YOUR CHATS:

Part 2: Text Files for Your Team

Save the following code blocks as `.txt` files and distribute them via your group chat or GitHub repository.

File 1: `master_context.txt`

(Give this to the AI whenever starting a new chat so it understands the global rules).

Plaintext

```
# MASTER PROJECT CONTEXT: 3D OPENGL OBSTACLE RUNNER
```

```
## 1. PROJECT RULES & CONSTRAINTS
```

- This is a 3D Computer Graphics university project built in Python using legacy OpenGL and GLUT.
- NO Shaders and NO Textures.
- Limited to basic shapes (``glutSolidCube``, ``gluCylinder``, ``gluSphere``, ``GL_QUADS``) and transformations (``glTranslatef``, ``glRotatef``, ``glScalef``).
- Must use ``glEnable(GL_DEPTH_TEST)``.
- The project must be contained entirely within a SINGLE ``py`` file.
- All code generated by the AI MUST be converted directly into Python.

```
## 2. PROJECT ARCHITECTURE & COMPLEXITY
```

To meet grading standards, this project is highly complex:

- ****Member 1 (World):**** Builds a 3D translating arcade tunnel, handles dynamic color interpolation for a day/night cycle, and manages a dynamic FOV "Warp Speed" camera effect.

- **Member 2 (Player):** Builds a hierarchical animated avatar (multi-part body with swinging limbs), implements smooth kinematics (lane switching, parabolic jumping), and handles a "Gravity Inversion" state where the player runs on the ceiling.
- **Member 3 (Obstacles):** Manages arrays of obstacles and power-ups using pattern-based spawning, implements morphing/pulsing obstacle animations, and writes dynamic 3D Axis-Aligned Bounding Box (AABB) collision logic.

3. "ZONED CODING" REPOSITORY RULES

Because three people are working in one file, the AI must strictly respect these comment zones and NEVER write code outside the requested zone:

- ZONE 0: ``# === GLOBAL VARIABLES ===``
- ZONE 1: ``# === MEMBER 1: WORLD & CAMERA ===``
- ZONE 2: ``# === MEMBER 2: PLAYER ===``
- ZONE 3: ``# === MEMBER 3: OBSTACLES & ITEMS ===``
- ZONE 4: ``# === MAIN GAME LOOP ===``

AI INSTRUCTION: Acknowledge you understand this master context. Ask the user which Member they are and what specific function they want to write.

File 2: [github_workflow.txt](#)

(Keep this in your repo's Readme or a shared folder so everyone knows the sequence).

Plaintext

SINGLE-FILE GITHUB WORKFLOW: THE "ZONED CODING" METHOD

Since we are forced to use a single `.py` file, we MUST follow this exact sequence to avoid Git merge conflicts. No one is allowed to push code outside their designated Zone.

****Phase 0: Initialization & Zoning (Member 1)****

1. Member 1 creates the GitHub repo with the base ``3D_OpenGL_Intro.py`` template.
2. Member 1 adds the empty Zone comment blocks (e.g., ``# === MEMBER 2: PLAYER ===``) to establish boundaries.
3. Member 1 pushes this foundation to the ``main`` branch.

****Phase 1: The Environment (Member 1)****

1. Member 1 creates branch ``feature-world``.

2. Codes the scrolling tunnel and camera setup strictly inside the "World" zone.
3. Merges `feature-world` into `main`. (The game now has an endless tunnel and camera).

****Phase 2: The Player (Member 2)****

1. Member 2 pulls the updated `main` branch and creates branch `feature-player`.
2. Codes the animated avatar, lane switching, and jumping states inside the "Player" zone.
3. Merges `feature-player` into `main`. (The game now has a tunnel and a controllable running avatar).

****Phase 3: The Threat (Member 3)****

1. Member 3 pulls the updated `main` branch and creates branch `feature-obstacles`.
2. Codes the obstacle manager, patterns, and AABB collision inside the "Obstacles" zone.
3. Merges `feature-obstacles` into `main`. (The core game loop is now fully playable).

****Phase 4: Novelty Integration (Parallel)****

1. With the core engine stable and zones respected, all members pull the latest `main`.
2. Everyone branches off simultaneously (`feature-warp`, `feature-gravity`, `feature-morph`).
3. Members code their specific novelty features inside their designated zones.
4. Branches are merged back into `main` one by one. Conflicts are avoided because all logic is contained within separate zones.

File 3: [member1_guide.txt](#)

(Member 1 uses this to guide their AI).

Plaintext

I am Member 1. My zone is `# === MEMBER 1: WORLD & CAMERA ===`. I need to build the environment and camera logic.

My Tasks & Execution Flow:

1. ****The Arcade Tunnel:**** I need to write a function that draws a grid floor and side pillars using `GL\_QUADS`, `glutSolidCube`, and `gluCylinder`. I need logic to translate these along the Z-axis to simulate movement, snapping them back to the start when they pass the camera.
2. ****Day/Night Cycle:**** I need a function that takes a global time variable and outputs interpolated RGB values to change the color of the tunnel over time.

3. ****Warp Speed Camera:**** I need to modify ``setupCamera()`` so the ``fovY`` variable gently expands as the game's global speed variable increases.

Let's start with Task 1. Please write the Python OpenGL code for a scrolling 3D tunnel. Ensure the variables used can be placed in the global variables zone.

File 4: [member2_guide.txt](#)

(Member 2 uses this to guide their AI).

Plaintext

I am Member 2. My zone is ``# === MEMBER 2: PLAYER ===``. I need to build the player avatar and kinematics.

My Tasks & Execution Flow:

1. ****Hierarchical Avatar:**** I need to write a ``draw_player()`` function using nested ``glPushMatrix`` and ``glPopMatrix`` to build a robotic runner with a body, arms, and legs.
2. ****Running Animation:**** I need to apply ``sin()`` wave math based on a time variable to ``glRotatef`` the limbs so it looks like it is running.
3. ****Kinematics:**** I need update functions for smooth X-axis lane switching (interpolating coordinates) and Y-axis jumping (using a parabolic gravity equation).
4. ****Gravity Inversion:**** I need logic that listens for a boolean. When true, it flips the player 180 degrees, sets their base Y to the ceiling, and inverts the jump math.

Let's start with Task 1 and 2. Please write the Python OpenGL code to draw a hierarchical, animated running avatar using basic shapes.

File 5: [member3_guide.txt](#)

(Member 3 uses this to guide their AI).

Plaintext

I am Member 3. My zone is ``# === MEMBER 3: OBSTACLES & ITEMS ===``. I need to build the collision engine and object manager.

My Tasks & Execution Flow:

1. ****Object Manager:**** I need lists of dictionaries to track active `obstacles` and `powerups` (storing type, X, Y, Z, active state).
2. ****Pattern Spawning:**** I need an update function that spawns objects in specific formations (e.g., a wall with a gap) at the far end of the Z-axis and moves them toward the camera.
3. ****Dynamic 3D AABB Collision:**** I need mathematical functions to detect if the player's 3D coordinates intersect with the obstacles or power-ups. The player's bounding box height must be adjustable via variables (for ducking/inverted states).
4. ****Morphing Logic:**** I need rendering logic that scales or swaps an obstacle's shape when its Z-distance from the camera drops below a certain threshold.

Let's start with Task 1 and 2. Please write the Python code for managing the lists of objects, translating them down the Z-axis, and spawning a simple 3-block hurdle formation.